

Homework — 1.2.2 Applications Generation — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Part A (14 marks)

1. [3] — 2 correct rows per mark: Application software = **E**; Utility software = **C**; Open source = **B**; Closed source = **G**; Linker = **F**; Loader = **A**. (*D and H are distractors: interpreter and cache.*)

2. [2] — Correct order:

Stage	Order
Code generation	3
Lexical analysis	1
Optimisation	4
Syntax analysis	2

All four correctly placed = 2 marks; two or three correctly placed = 1 mark.

3. [3] — 1 mark per row:

- (a) Lexical analysis: the source code is converted into a series of **tokens**; whitespace and comments are removed; a **symbol table** of identifiers (variables, subroutines) is created for later stages to look up.
- (b) Syntax analysis: the tokens are checked against the **grammar rules** of the language (e.g. by building an abstract syntax tree); errors are reported if the rules are broken.
- (c) Optimisation: the code is refined/rearranged so it runs **faster and/or uses less memory** while producing exactly the same result.

4. [3] — 1 mark per statement correctly fixed:

- 1: The **linker** combines the compiled modules and library code into a single executable. (*The loader's job is to copy the finished executable from secondary storage into RAM and prepare it to run.*)
- 2: That describes **dynamic** linking. With **static** linking, the library code is **copied into the executable at compile/link time**, so updating the library does not update already-built programs.
- 3: That is an advantage of **static** linking. A **dynamically** linked program depends on the separate library file (e.g. a .dll) being present at run time.

5. [3] — Open source advantages (1 each, max 2): free / no licence cost; source code can be viewed and modified to suit the school; large community support; not tied to one vendor. Closed source advantage (1): official/dedicated vendor support and warranties; often more polished / fully tested / regular professional updates. Max 3.

Part B (6 marks)

6. [3] — Any three (1 each): CPU checks the interrupt register at the end of the current FDE cycle; finishes the current instruction / compares priorities; pushes the registers (PC, ACC) onto the stack; loads the address of and runs the correct ISR; pops the registers off the stack to resume. The stack is LIFO so nested/higher-priority interrupts resume the earlier task in the correct order. Max 3.

7. [1] — Any one (1): RISC has a small/simple instruction set with fixed-length instructions executed in (typically) one cycle, whereas CISC has many complex variable-length instructions that may take several cycles; RISC relies more on software/compiler and pipelining; CISC does more per instruction. Max 1.

8. [2] — Von Neumann uses a **single shared memory and bus** for both instructions and data (1), whereas Harvard uses separate memories/buses for instructions and data (1).

Total: 14 + 6 = 20